STEAMGEARS.EU · RESEARCH

Sikkerhed: lækkede nøgler i git-historikken

rotation og oprydning

Dato: 2. november 2026 · **Status:** rotation ikke udført (kræver ejeren) · **Alvor:** høj

1. Hvad der blev fundet

Den 30. oktober blev en `.env` committet sammen med Command Center v8.0 (commit `3332881`) og en backup-kopi (`.env.backup-20261030-214616`, commit `0883e92`). Filen blev fjernet igen i `800a711` («Remove Command Center v8.4 and committed .env files»), men **fjernelsen rydder ikke historikken**.

Blobben ligger stadig i `main`s historik og kan hentes af enhver med adgang til repoet:

```
git show e5ee31b:.env                       # virker fra ethvert klon af main
git show e5ee31b:.env.backup-20261030-214616
```

Den indeholder fire nøgler. Værdierne gengives ikke her — kun navn og længde, målt 2/11:

Nøgle	Længde	Vurdering
JWT_SECRET	34 tegn	Ser ud som en rigtig hemmelighed, ikke en pladsholder
WP_SITE_URL	21 tegn	Ikke hemmelig
WP_USER	14 tegn	Brugernavn, ikke hemmeligt
WP_APP_PASSWORD	16 tegn	For kort til et application password (de er 24 tegn med mellemrum), så det er formentlig en rigtig adgangskode

`e5ee31b` er en ancestor af `main` (bekræftet med `git merge-base --is-ancestor`), og der er **77 commits** mellem fjernelsen og `3d9725e`. Det betyder, at nøglerne har været udstillet i hele perioden — og stadig er det for alle, der har klonet repoet, og for eventuelle forks.

2. Hvad det betyder

- **Adgangsniveauet er det vigtigste.** Er `WP_APP_PASSWORD` et rigtigt application password for en administratorbruger, giver det REST-skrivning til sitet — det er præcis den adgang agenten mangler (401).
- **Et offentligt repo gør det værre.** `.gitignore` kalder selv repoet «this public repo». Er det offentligt, skal nøglerne betragtes som kompromitterede, uanset om de er slettede i nyere commits.
- **Historik-rensning alene løser ikke problemet.** Nøgler, der har været udstillet, skal skiftes uanset hvad.

3. Rotations-tjekliste (ejeren, i denne rækkefølge)

Rækkefølgen er valgt, så adgangen ikke bryder sammen undervejs: ny legitim adgang først, derefter lukning af den gamle.

1. **Kortlæg hvem der bruger WP-adgangen.** `seo_agent.py` (env), CI-secret `WP_APP_PASS`, MCP (`MCP_ACCESS_TOKEN`) og Command Centerets egen `.env`. Alle fire steder skal have den nye værdi.
2. **Opret nyt application password** i wp-admin → Brugere → Profil → Application Passwords. Giv det et navn, der siger hvad det bruges til (fx `ci-seo-audit`), og brug en bruger med de rettigheder opgaven kræver — ikke nødvendigvis administrator.
3. **Tilbagekald alle gamle application passwords** samme sted (Revoke). Det lukker den lækkede nøgle, hvis den er et application password.

4. **Skift brugerens login-adgangskode**, hvis den 16-tegns værdi var en rigtig adgangskode. Slå to-faktor til, hvis det er muligt på Simply.
5. **Rotér JWT_SECRET** i Command Center og genstart appen. Alle eksisterende sessioner bliver ugyldige — det er meningen.
6. **Rotér GEMINI_API_KEY** i Google AI Studio (opret ny, slet den gamle) og opdatér repo-secret `GEMINI_API_KEY` under Settings → Secrets and variables → Actions.
7. **Rotér PINTEREST_ACCESS_TOKEN**, hvis den har været brugt lokalt.
8. **Verificér at det gamle virker ikke længere** (se §4), og at CI er grøn.

4. Verifikation efter rotation

```
# Gammel WP-adgang skal give 401, ny skal give 200
curl -s -o /dev/null -w '%{http_code}\n' -u 'GAMMEL_BRUGER:GAMMEL_NØGLE' \
  'https://steamgears.eu/wp-json/wp/v2/users/me'
curl -s -o /dev/null -w '%{http_code}\n' -u 'NY_BRUGER:NY_NØGLE' \
  'https://steamgears.eu/wp-json/wp/v2/users/me'

# Gammel Gemini-nøgle skal afvises
curl -s -o /dev/null -w '%{http_code}\n' \
  'https://generativelanguage.googleapis.com/v1beta/models?key=GAMMEL_NØGLE'
```

Forventet: `401` (eller `403`) på de gamle og `200` på de nye. Kør også `seo_agent.py` lokalt med den nye nøgle, og se at `seo_audit.yml` bliver grøn i Actions.

5. Historik-rensning — to veje

A. Rotér og lev med historikken (anbefalet)

Når nøglerne er skiftet, er de gamle værdier værdiløse. Historikken indeholder stadig strengen, men den kan ikke bruges til noget. Fordelen er, at ingen klon skal laves om, og at ingen commits skifter SHA. Ulempen er, at fremtidige læsere af historikken kan tro, at nøglen stadig gælder.

B. Skriv historikken om (kræver ejerens beslutning)

```
# I et frisk klon, ikke i denne arbejdskopi:
git filter-repo --path .env --path .env.backup-20261030-214616 --invert-paths
git push --force-with-lease origin main
```

Konsekvenser, der skal accepteres først:

- **Alle** commits efter det ændrede punkt får nye SHA'er. Åbne pull requests og alle eksisterende klon skal laves om.
- GitHub beholder ikke-slettede objekter, indtil deres garbage collection kører; forks beholder deres egen kopi.
- Force-push er ikke noget agenten gør. Det er en ejer-beslutning, og den skal tages eksplicit.

Anbefaling: gør A nu (rotation), og B kun hvis repoet er eller bliver offentligt, eller hvis en sikkerhedsrevision kræver det.

6. Sådan undgås gentagelse

- `.gitignore` dækker allerede `.env`, `.env.*` (med `.env.example` som undtagelse), `*.key`, `*.pem` og `steamgears_auth.json`. Det blev kontrolleret 2/11 med `git check-ignore`.
- Brug `.env.example` som skabelon — den indeholder kun navne og pladsholdere.
- Slå **secret scanning / push protection** til i GitHub, hvis det ikke er gjort: Settings → Code security.

- Rotations-tjeklisten i §3 bør gentages, hvis en nøgle nogensinde har været i en commit — også i en slettet fil.